

Live Allocation-Aware Resource Advising on Chameleon: Corpus Triage, Blazar Integration, and Targeted Availability Fetch

Vivek Rai
Chameleon Fellowship
chi-edge-advisor / benchmark_v4

Abstract—We report three connected pieces of work on the Chameleon resource advisor. First, a deterministic ingestion pipeline over the 205-artifact Trovi catalog, which establishes that only 95 artifacts contain Chameleon provisioning code at all and only 90 are usable for resource recommendation. Second, replacement of the advisor’s non-functional `python-chi` availability path with a direct Blazar REST client covering all seven Chameleon sites, adding reservation time windows and a previously invisible third node state. Third, a resource catalog mapping node types to sites, enabling targeted fetches that cut availability latency by up to $10.8\times$. We also record the architectural decision to abandon the hierarchical artifact router in favour of a flat, tag-indexed JSON store, which measurement shows is not merely simpler but more accurate.

Index Terms—testbed provisioning, resource allocation, Blazar, retrieval, reproducibility

I. FROZEN-MEASUREMENT BASELINE

All downstream work is diffed against a pinned baseline so corpus expansion cannot silently perturb the benchmark. Capturing it exposed that the committed baseline was already stale: it recorded 406 empty / 150 scored cells, while the `runs/` files committed in the *same* commit rescore to 314 / 242. There was, in effect, no valid baseline before this work.

TABLE I
PINNED PARITY ARTIFACT (556 CELLS, 242 NON-EMPTY)

Artifact	sha256 (truncated)
<code>run_scores.csv</code>	252e42bf...1387525
<code>gate_report.json</code>	d51facdb...71f3da6e
content manifest (63 files)	e26c9537...cac9505a

The gold gate passes 50/50 and the pin re-verifies byte-identically. Physical separation does the enforcing: nothing the pipeline produces is written to `benchmark_v4/grounding/`, because `run_bench.py:90` and `tier_assign.py:33` glob those paths and the frozen provenance hash depends on them.

II. CORPUS TRIAGE: THE REAL N

The 205 catalog artifacts were fetched (180 cloned, 20 sparse-checkout above a 200 MB cap, 7 unfetchable) and clas-

sified by AST-based detection of the measured `python-chi` call surface.

TABLE II
TRIAGE OUTCOME OVER 205 TROVI ARTIFACTS

Class	<i>n</i>	Consequence
no-chameleon-code	108	unusable: no provisioning grammar
instructional	49	tutorial-shaped grounding
evidence	46	research-shaped grounding
excluded	2	policy (v1 overlap, explicit)
usable	95	real <i>N</i> , not 205

This is the headline planning number. 53 % of the catalog carries no Chameleon provisioning code, typically because the GitHub link points at upstream paper code while the provisioning lives in the Trovi body. Ranking the 95 by how much of a *groundable* resource recommendation can be extracted narrows it further: 39 tier-A and 51 tier-B, so **90 artifacts** are worth ingesting. Any plan sized on 205 is roughly $2\times$ over.

A secondary finding cuts the other way. The new corpus contains **52 artifacts using `add_node_reservation`**, where the existing workspace had the bare-metal grammar zero times outside `forbidden_calls` trap lists. The corpus supplies exactly the positive examples the emitter lacks.

Detection was cross-checked against an independent regex sweep over all 102 inspectable negatives: *one* true false negative, caused by a 9.4 MB notebook skipped by a byte cap. Notebook size is driven by embedded output images rather than code, so the cap now applies to extracted source.

III. ARCHITECTURE: TREE $\rightarrow$ TAG-INDEXED STORE

The advisor was originally designed around a hierarchical router: `site` $\rightarrow$ `use-case` $\rightarrow$ `artifact` (`RouterTree`). We abandoned it for a flat store with JSON tag records. This was not simplification for its own sake; the hierarchy measurably retrieves worse.

The tree loses 17.8 points at the artifact level. The cause is structural: every hard gate is a place to be irrecoverably wrong. A use-case misclassification at L1 removes the correct

TABLE III
ROUTER ABLATION A7, ARTIFACT HIT RATE (45 COVERED ITEMS)

Level	Tree	Flat + tags
L0 site	100.0 %	n/a
L1 use-case @2	73.3 %	n/a
L2 artifact @3	68.9 %	86.7 %
chunk-level	68.9 %	86.7 %

artifact from consideration before L2 ever runs, and the L1 gate is only 73.3 % accurate. Flat scoring lets a weak signal on one facet be outvoted rather than becoming fatal.

The corpus is therefore stored as flat tagged records, not a hierarchy: `catalog.jsonl`, `manifest.tsv` (18 columns), and `resource_catalog.json`. Two orthogonal tag vocabularies are emitted rather than one taxonomy, because the advisor performs a *mapping*: 23 workload tags on the query side (what the user wants) and 13 resource tags on the answer side (what that implies about hardware). Resource tags derive only from observed code, never prose, since that half is what the emitter renders. TSV was chosen over JSON for the scan table on a measured 57 versus 238 tokens per row, paid 200 times.

IV. LIVE STATE: BLAZAR, NOT THE REFERENCE API

The advisor’s availability layer did not work. The public reference API is **static-only** and exposes zero CHI@Edge inventory; its deeper endpoints currently return 500/502. The `python-chi` backend failed at its own site-discovery call and saw only one site. We replaced it with direct REST: `Keystone /auth/tokens → service catalog → Blazar → {resource} + /allocations`.

Three findings shaped the design.

(1) **Blazar has no server-side filter.** Measured, not assumed: `?node_type=`, `?q=`, `?filter=` and `?query=` all return HTTP 200 with *all* 132 hosts. Narrowing is therefore possible only by contacting fewer *sites*.

(2) **Reservations do not name their hosts.** Reading `/leases` alone finds nothing reserved and reports every host free, even at a site with an active lease. The mapping lives on a separate `/allocations` endpoint. This single correction moved the answer from “587 free of 587” to Table IV.

(3) **There is a third state.** `reservable=false` is neither free nor reserved. A node in maintenance is unbookable, and reporting it free is precisely the failure that yields a spec that cannot be submitted. **111 of 655 nodes** were in that state, previously counted as free.

CHI@Edge required separate discovery: it answers `/devices`, not `/os-hosts`, and its machine type lives in `machine_name` (`device_type` is uniformly “container”). Every GPU class of interest (A100, V100, H100) was fully reserved at capture time, leaving only older silicon free. That contention is exactly what the advisor exists to reason about. Per-node reservation windows are also now recorded, so the advisor can answer “free for how long” and “if not, when” rather than only “free now”.

TABLE IV
LIVE CHAMELEON FLEET, ALL SEVEN SITES

Site	Resource	Free	Busy	Down	Total
CHI@Edge	devices	35	0	33	68
CHI@NCAR	os-hosts	35	8	1	44
CHI@NRP	os-hosts	0	0	4	4
CHI@NU	os-hosts	6	0	2	8
CHI@TACC	os-hosts	112	181	21	314
CHI@UC	os-hosts	18	93	21	132
KVM@TACC	os-hosts	2	54	29	85
Total		208	336	111	655

V. TARGETED FETCH

Because filtering must happen at site granularity, we persist a node-type $\rightarrow$ site catalog refreshed by a full sweep. The distribution makes this worthwhile: of 41 distinct types, **37 live on exactly one site** (2 on two, 2 on three), a mean of 1.15 sites per type.

TABLE V
AVAILABILITY FETCH LATENCY, MEASURED

Request	Sites	Nodes	Time	Speedup
Full sweep (baseline)	7	655	34.9 s	1.0×
<code>compute_skylake</code>	3	57	18.9 s	1.8×
<code>gpu_rtx_6000</code>	1	37	7.5 s	4.7×
Pi camera workload	1	46	3.2 s	10.8×

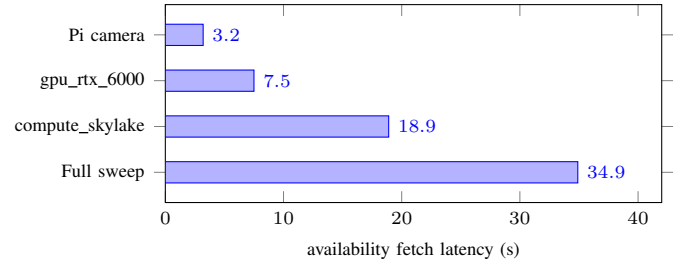


Fig. 1. Targeted fetch versus full sweep. The saving scales with how few sites carry the requested type.

The pipeline was reordered to exploit this. Retrieval reads only the workload string and the artifact registry, so it can run *before* any network call and tell the availability layer which hardware to ask about:

RETRIEVE $\rightarrow$ READ $\rightarrow$ REASON $\rightarrow$ VALIDATE $\rightarrow$ EMIT

End to end, the pi-camera workload now contacts one site and sees 46 devices instead of 655, and still validates PASS.

Two guards prevent the optimisation from causing wrong answers, since pruning incorrectly is worse than not pruning. An unrecognised type falls back to the full sweep rather than pruning to an empty set. If the reasoner names a type outside the candidate set, one targeted re-fetch runs (measured 3.5s) instead of returning empty availability, which would fail `device_free_in_window` and silently turn a good recommendation into a non-zero exit.

VI. MULTI-SITE EMISSION

Recommendation now carries `site` and `api_family`, copied from the catalog rather than inferred, and the emitter dispatches to three genuinely distinct grammars.

TABLE VI
EMITTER DISPATCH, VERIFIED

Family	Types	Reservation call	Boot
edge	7	<code>add_device_reservation</code>	Container
baremetal	31	<code>add_node_reservation</code>	Server
kvm	3	<code>add_flavor_reservation</code>	Server

Stated plainly: the bare-metal template is written from the `python-chi` API, not from anything observed locally, because `add_node_reservation` and `create_server` appear zero times as real calls in this workspace. Both non-edge templates carry an UNVALIDATED GRAMMAR banner and are excluded from live dry-check. They are also unreachable in practice today: all five registry artifacts are `site="CHI@Edge"`, so routing never selects non-edge hardware. The dispatch is ready; grounding it is the next task.

VII. REMAINING WORK

- 1) **Ingest the 90 tier-A/B artifacts** into the registry. This is the blocking item: multi-site emission is structurally complete but ungrounded, so a GPU-training query still routes to a Raspberry Pi.
- 2) **Teach reservation from artifacts** — what to reserve and how — grounding the bare-metal and KVM grammars against the 52 newly available `add_node_reservation` examples.
- 3) **Schedule the catalog refresh.** A 168 h TTL safety net exists but nothing schedules the sweep, so it can stall a user call by ~ 35 s.
- 4) **Phase-1 adversarial audit** of the 10 selected artifacts, then the 25-artifact stratified phase 2. Seven strata have only one member.
- 5) **Recover device profiles for non-edge** types, and resolve the 4 Trovi-hosted artifacts unreachable while the Trovi API returns 404.

VIII. CONCLUSION

The corpus is half the size it appeared: 95 of 205 artifacts carry provisioning code, and 90 are worth ingesting. The advisor now reads genuinely live state across all seven sites, including a maintenance state covering 17% of the fleet that was previously reported as free. Targeted fetching makes that state cheap enough to consult on every query, at up to $10.8\times$ less latency. The architectural lesson worth carrying forward is the router ablation: the hierarchical design was intuitive and measurably worse, because every hard gate is a chance to be irrecoverably wrong.

Throughout, the frozen 556-cell measurement surface verified byte-identical and the advisor test suite remained at 38/38.